

Deep Learning, Introduction and Advanced Concepts

2. Optimization and Architectures

Louis Bagot

January 5, 2026

Reminders from last time

1. We want to **approximate a function**
2. For this we use a (Deep) **Neural Network**
3. We train it with **backpropagation and gradient descent**

Reminders 1. Approximating a function: Motivations

All these problems can be seen as *looking for a function* $g : \mathcal{X} \rightarrow \mathcal{Y}$:

Text	Image	Audio & Signal	Decision
document analysis	image recognition	audio recognition	chess
sentiment analysis	segmentation	forecasting	starcraft
translation	style transfer	translation	minecraft
text generation	image generation	audio generation	robotics

Reminders 1. Approximating a function: Objective

Reminders 1. Approximating a function: Objective

Supervised Learning:

- Given examples of g as dataset $\mathcal{D} = \{\mathbf{x}_i, g(\mathbf{x}_i) = \mathbf{y}_i\}_{i=1}^N$,

Reminders 1. Approximating a function: Objective

Supervised Learning:

- Given examples of g as dataset $\mathcal{D} = \{\mathbf{x}_i, g(\mathbf{x}_i) = \mathbf{y}_i\}_{i=1}^N$,
- **Train** “parametric” function

$$f_{\theta} \approx g : \mathcal{X} \rightarrow \mathcal{Y}$$

Notes for this slide

Reminders 1. Approximating a function: Objective

Supervised Learning:

- Given examples of g as dataset $\mathcal{D} := \{(\mathbf{x}_i, y(\mathbf{x}_i)) = y_i\}_{i=1}^n$
- Train "parametric" function

$$f_{\theta} \approx g : \mathcal{X} \rightarrow \mathcal{Y}$$

"Parametric" refers to the fact that the function we learn, f , is controlled by *parameters* θ – when the parameters change, the shape of the function changes (like a polynomial with a, b, c or a linear model with w_i). The objective is to find the parameters that best approximate the target function g .

Reminders 1. Approximating a function: Objective

Supervised Learning:

- Given examples of g as dataset $\mathcal{D} = \{\mathbf{x}_i, g(\mathbf{x}_i) = \mathbf{y}_i\}_{i=1}^N$,
- **Train** “parametric” function

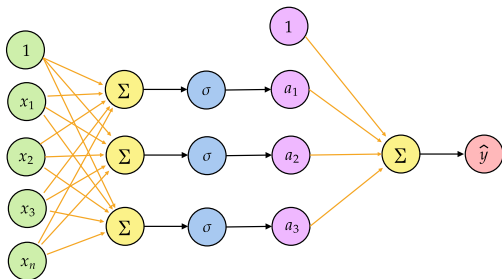
$$f_{\theta} \approx g : \mathcal{X} \rightarrow \mathcal{Y}$$

- **Formal Objective:** minimize *Loss* $\mathcal{L}$, distance between g and f_{θ} , e.g. MSE:

$$\text{find } \arg \min_{\theta} \mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_i (\mathbf{y}_i - f_{\theta}(\mathbf{x}_i))^2$$

Reminders 2. Using Neural Networks for f_{θ}

What function class to choose for parametric function f_{θ} ?

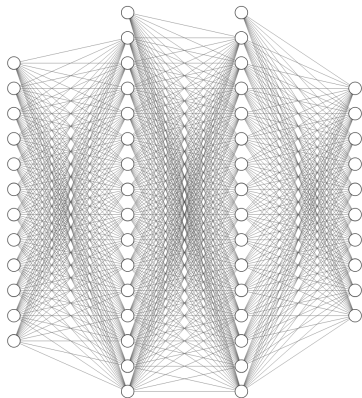


Classical NN: sequence of linear regressions + activation function σ .

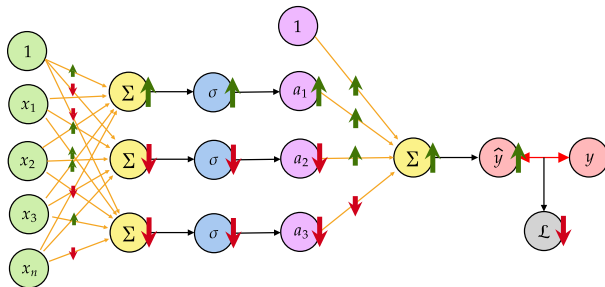
Parameters θ (orange): weights of the linear regressions.

Reminders 2. Using Neural Networks for f_θ

Notion of Layers of abstraction



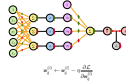
Reminders 3. Training with Backpropagation



$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}}$$

Notes for this slide

Reminders 3. Training with Backpropagation



Great resource for visualizing backprop: [this website](#).

Deep Learning Course 2: Optimization and Architectures

Still lots of works to do to make it work in practice!

1. Gradient Descent and Optimization
2. Stabilization
3. Architectures

Deep Learning Course 2: Optimization and Architectures

Still lots of works to do to make it work in practice!

1. **Gradient Descent and Optimization**
2. Stabilization
3. Architectures

From Gradient Descent to *Mini-batch* Gradient Descent

From Gradient Descent to *Mini-batch* Gradient Descent

- We want to minimize $\mathcal{L}$ over the entire dataset

$$\arg \min_{\theta} \mathcal{L}(f_{\theta}, \mathcal{D}).$$

- Computing $\partial \mathcal{L}(f_{\theta}, \mathcal{D}) / \partial w_{ij}$ for Gradient Descent is **too expensive**
→ $\mathcal{D}$: millions or billions of datapoints for a single update!

From Gradient Descent to *Mini-batch* Gradient Descent

- Break $\mathcal{D}$ down into much smaller *mini-batches* $\mathcal{B} = \{(\mathbf{x}_s, \mathbf{y}_s)\}_{s=1}^n$
 - ▶ e.g. of size $n = 32$ samples, taken randomly from $\mathcal{D}$

From Gradient Descent to *Mini-batch* Gradient Descent

→ Break $\mathcal{D}$ down into much smaller *mini-batches* $\mathcal{B} = \{(\mathbf{x}_s, \mathbf{y}_s)\}_{s=1}^n$

▶ e.g. of size $n = 32$ samples, taken randomly from $\mathcal{D}$

1. Compute $\mathcal{L}(f_{\theta}, \mathcal{B})$ e.g. $\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_s (\mathbf{y}_s - f_{\theta}(\mathbf{x}_s))^2$
2. Compute $\partial \mathcal{L}(f_{\theta}, \mathcal{B}) / \partial w_{ij} \approx \partial \mathcal{L}(f_{\theta}, \mathcal{D}) / \partial w_{ij}$
3. Perform *Mini-batch* Gradient Descent

Types of Gradient Descent

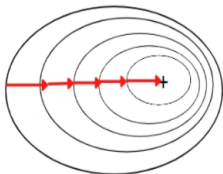
Size of the mini-batch = trade-off between

Accurate but expensive

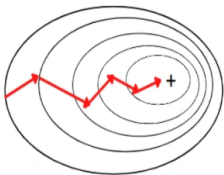


Quick but noisy

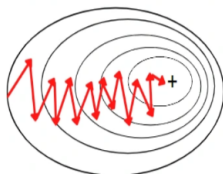
Gradient Descent
($n=N$)



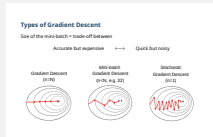
Mini-batch
Gradient Descent
($n < N$, e.g. 32)



Stochastic
Gradient Descent
($n=1$)



Notes for this slide

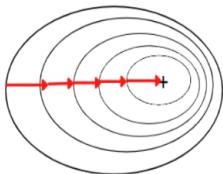


ellipses plots from [this blog post](#). The ellipses corresponds to the contour plot of the full loss function ($n=N$), so only Gradient Descent points in the direction of the closest local minima; MBGD and SGD are approximations of the GD ("true") gradient.

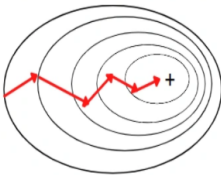
When using big accurate steps (GD), the learning rate η can afford to be high, but with noisier steps (MB GD and SGD) the learning rate generally needs to be lower.

Types of Gradient Descent

Gradient Descent
($n=N$)

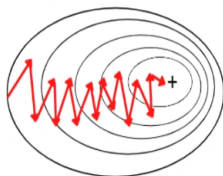


Mini-batch
Gradient Descent
($n < N$, e.g. 32)



always done in Deep Learning

Stochastic
Gradient Descent
($n=1$)



Improving Mini-Batch Gradient Descent

With Mini-Batch Gradient Descent, we have introduced a lot of variance and instabilities

→ How can we make gradient descent more reliable?

Notes for this slide

Improving Mini-Batch Gradient Descent

With Mini-Batch Gradient Descent, we have introduced a lot of variance and instabilities.

→ How can we make gradient descent more reliable?

Instability: when small changes to the training setup (data, parameters, initialization, updates...) make huge changes in the outcome of the training. Ideally, we want the training to be *robust* to have limited impact on the final (good) performance.

With mini-batch gradient descent, the small updates in wrong directions can take the model completely off-course.

Momentum and Adaptive Learning Rates

With MBGD, we take noisy steps instead of confident ones.

→ lower the learning rate: many small steps instead of big confident ones

Momentum and Adaptive Learning Rates

With MBGD, we take noisy steps instead of confident ones.

- lower the learning rate: many small steps instead of big confident ones
- the training can be slow, noisy and often get stuck in *saddle points*
 - imagine a ball getting stuck in a small dip
 - introduce inertia: **momentum** !

Notes for this slide

Momentum and Adaptive Learning Rates

With MBGD, we take noisy steps instead of confident ones.

→ lower the learning rate: many small steps instead of big confident ones

• the training can be slow, noisy and often get stuck in saddle points

→ imagine a ball getting stuck in a small dip

• introduce inertia: ~~momentum~~!

saddle points: regions where the loss landscape is nearly flat so the gradient is near zero: no parameter changes, no training. A region where it looks like no small changes would change the performance.

For a visualization of momentum see e.g. [this short](#)

Gradient Descent with Momentum

Instead of the (Mini-Batch) Gradient Descent update for parameter θ_i at time $t + 1$,

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta \partial \mathcal{L} / \partial \theta_i^t$$

Each update is

$$\begin{aligned}\theta_i^{t+1} &\leftarrow \theta_i^t - \eta m_i^t \\ m_i^t &= \beta m_i^{t-1} + (1 - \beta) \partial \mathcal{L} / \partial \theta_i^t\end{aligned}$$

- a bit of the new gradient $\partial \mathcal{L} / \partial \theta_i^t$
- a lot of the old gradients m_i^t (generally $\beta > 0.99$)

Adaptive Learning Rates

Single learning rate η : very tricky hyperparameter to find

- too low $\rightarrow$ very long wait
- too high $\rightarrow$ unstable, non-converging
- should not necessarily be the same for all parameters

Adaptive Learning Rates

Single learning rate η : very tricky hyperparameter to find

- too low $\rightarrow$ very long wait
- too high $\rightarrow$ unstable, non-converging
- should not necessarily be the same for all parameters

Adaptive learning rates: scaling η with v_i^t

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta v_i^t m_i^t$$

- if a parameter changes often, take careful steps (small v_i^t)
- otherwise, rarer updates $\rightarrow$ needs bigger updates (high v_i^t)

Notes for this slide

Adaptive Learning Rates

Single learning rate η : very tricky hyperparameter to find

- too low $\rightarrow$ very long wait
- too high $\rightarrow$ unstable, non-converging
- should not necessarily be the same for all parameters

Adaptive learning rates: scaling η with v_i^t

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta v_i^t$$

- if a parameter changes often, take careful steps (small v_i^t)
- otherwise, rarer updates $\rightarrow$ needs bigger updates (high v_i^t)

An example of adaptive learning rate is Root Mean Squared Propagation (RMSProp):

- the gradient is scaled by $1/\sqrt{v_i^t}$
- with $v_i^t = \beta_2 v_i^t + (1 - \beta_2)(\partial\mathcal{L}/\partial\theta_i^t)^2$
- so $\rightarrow$ the more active the gradient $\implies$ the higher the square gradient $(\partial\mathcal{L}/\partial\theta_i^t)^2$
 $\implies$ the bigger $v_i^t \implies$ the smaller the update:

$$\theta_i^{t+1} \leftarrow \theta_i^t + \eta m_i^t / \sqrt{v_i^t}$$

The square root comes from math details due to squaring gradients.

Optimizers

Backpropagation gives us $\partial\mathcal{L}/\partial\theta_i^t$.

- Gradient Descent wants to follow the negative gradient
- Using momentum m controls *direction*
- Using adaptive learning rate ν controls *speed*

The *Optimizer* is the algorithm that chooses the next θ_i^{t+1} given θ_i^t and the loss gradient $\partial\mathcal{L}/\partial\theta_i^t$.

Optimizers

Backpropagation gives us $\partial\mathcal{L}/\partial\theta_i^t$.

- Gradient Descent wants to follow the negative gradient
- Using momentum m controls *direction*
- Using adaptive learning rate ν controls *speed*

The *Optimizer* is the algorithm that chooses the next θ_i^{t+1} given θ_i^t and the loss gradient $\partial\mathcal{L}/\partial\theta_i^t$.

Adam has been the most popular optimizer for nearly a decade! Combines momentum + adaptive learning rates (Adam = Adaptive moment estimation).

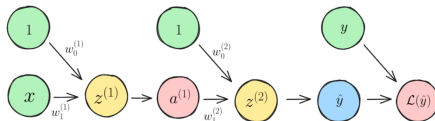
Deep Learning Course 2: Optimization and Architectures

Still lots of works to do to make it work in practice!

1. Gradient Descent and Optimization
2. **Stabilization**
3. Architectures

Vanishing and Exploding Gradients in Deep Learning

Remember Backpropagation:

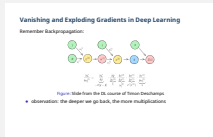


$$\frac{\partial \mathcal{L}}{\partial w_1^{(1)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}}}_{-2(y - \hat{y})} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z^{(2)}}}_1 \cdot \underbrace{\frac{\partial z^{(2)}}{\partial a^{(1)}}}_{w_1^{(2)}} \cdot \underbrace{\frac{\partial a^{(1)}}{\partial z^{(1)}}}_{\sigma'(z^{(1)})} \cdot \underbrace{\frac{\partial z^{(1)}}{\partial w_1^{(1)}}}_x$$

Figure: Slide from the DL course of Timon Deschamps

- observation: the deeper we go back, the more multiplications

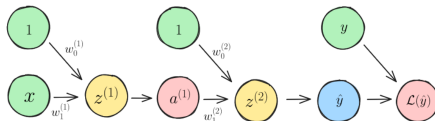
Notes for this slide



The neuron *activation* is the outcome of applying the activation function, generally the “layer value” (red a in the plot).

Vanishing and Exploding Gradients in Deep Learning

Remember Backpropagation:

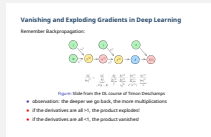


$$\frac{\partial \mathcal{L}}{\partial w_1^{(1)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}}}_{-2(y - \hat{y})} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z^{(2)}}}_1 \cdot \underbrace{\frac{\partial z^{(2)}}{\partial a^{(1)}}}_{w_1^{(2)}} \cdot \underbrace{\frac{\partial a^{(1)}}{\partial z^{(1)}}}_{\sigma'(z^{(1)})} \cdot \underbrace{\frac{\partial z^{(1)}}{\partial w_1^{(1)}}}_x$$

Figure: Slide from the DL course of Timon Deschamps

- observation: the deeper we go back, the more multiplications
- if the derivatives are all >1, the product explodes!
- if the derivatives are all <1, the product vanishes!

Notes for this slide



The neuron *activation* is the outcome of applying the activation function, generally the “layer value” (red a in the plot).

Addressing Vanishing and Exploding Gradients

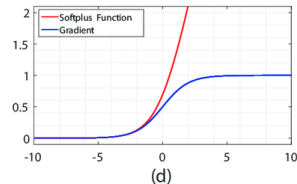
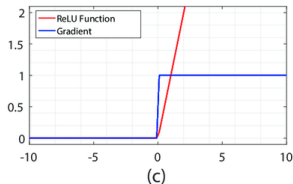
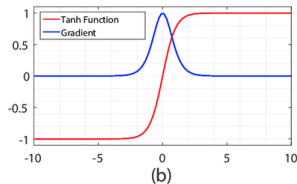
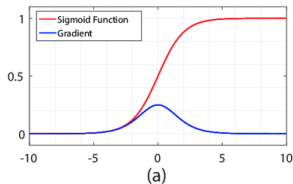
Activation Functions

Derivatives of the activation function σ show up at every layer

→ if $\sigma' < 1$ most of the time, vanishing gradient!

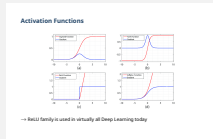
- sigmoid function has this issue, doesn't work well in practice

Activation Functions



→ ReLU family is used in virtually all Deep Learning today

Notes for this slide



“ReLU family” contains ReLU, LeakyReLU, GELU, Swish... look them up! They’re all small variations on ReLU.

What we want from activation functions

- Non-linear
- Differentiable
- Fast to compute
- Normalized (zero-centered, std 1)
- Non-saturating (gradient $\neq 0$ most of the time)

Addressing Vanishing and Exploding Gradients

Parameter initialization

Assuming activation function is nice, worry about parameter initialization $\theta^{t=0}$:

→ how should we initialize our parameters?

Notes for this slide

We are essentially asking: for the problem of rolling the ball down the hill (gradient descent), where should the ball start? Intuition: as low as possible and wherever the hill is smooth and leads to the lowest point.

If parameters are not zero in average, then applying them layer after layer will drift the output in a direction ($\pm\infty$ or the max of the activ. func.)

Since the derivative of next layer activation w.r.t the current is θ_i , if the variance in θ is small (the weight values are small overall), the derivative is small and we have a vanishing gradient issue (same for high values leading to exploding).

For more on initialization see [this tutorial](#)

Addressing Vanishing and Exploding Gradients

Parameter initialization

Assuming activation function is nice, worry about parameter initialization $\theta^{t=0}$:

→ how should we initialize our parameters?

- if weights are not zero-centered, gradual drift at each layer
- if weight variance is too small, vanishing gradient
- if weight variance is too big, exploding gradient

Notes for this slide

Addressing Vanishing and Exploding Gradients Parameter Initialization

Assuming activation function is nice, worry about parameter initialization $\theta^{(0)}$:

→ how should we initialize our parameters?

- if weights are not zero-centered, gradual drift at each layer
- if weight variance is too small, vanishing gradient
- if weight variance is too big, exploding gradient

We are essentially asking: for the problem of rolling the ball down the hill (gradient descent), where should the ball start? Intuition: as low as possible and wherever the hill is smooth and leads to the lowest point.

If parameters are not zero in average, then applying them layer after layer will drift the output in a direction ($\pm\infty$ or the max of the activ. func.)

Since the derivative of next layer activation w.r.t the current is θ_i , if the variance in θ is small (the weight values are small overall), the derivative is small and we have a vanishing gradient issue (same for high values leading to exploding).

For more on initialization see [this tutorial](#)

Addressing Vanishing and Exploding Gradients

Parameter initialization

Assuming activation function is nice, worry about parameter initialization $\theta^{t=0}$:

→ how should we initialize our parameters?

- if weights are not zero-centered, gradual drift at each layer
- if weight variance is too small, vanishing gradient
- if weight variance is too big, exploding gradient
- Activation Function-specific initialization, e.g. He/Kaiming for ReLU:

$$\theta \sim \mathcal{U}(-a, a) \text{ with } a = \sqrt{6/n_{\text{in}}}.$$

Notes for this slide

Addressing Vanishing and Exploding Gradients Parameter Initialization

Assuming activation function is nice, worry about parameter initialization $\theta^{(0)}$:

→ how should we initialize our parameters?

- if weights are not zero-centered, gradual drift at each layer
- if weight variance is too small, vanishing gradient
- if weight variance is too big, exploding gradient

• Activation Function-specific initialization, e.g. He/Normalizing for ReLU:

$$\theta \sim \mathcal{U}(-a, a) \text{ with } a = \sqrt{2/n_{\text{in}}}$$

We are essentially asking: for the problem of rolling the ball down the hill (gradient descent), where should the ball start? Intuition: as low as possible and wherever the hill is smooth and leads to the lowest point.

If parameters are not zero in average, then applying them layer after layer will drift the output in a direction ($\pm\infty$ or the max of the activ. func.)

Since the derivative of next layer activation w.r.t the current is θ_i , if the variance in θ is small (the weight values are small overall), the derivative is small and we have a vanishing gradient issue (same for high values leading to exploding).

For more on initialization see [this tutorial](#)

Addressing Vanishing and Exploding Gradients

Normalization

If we want to restrict the neuron activations to convenient intervals (e.g. centered around 0)... **simply normalize them!**

Addressing Vanishing and Exploding Gradients

Normalization

If we want to restrict the neuron activations to convenient intervals (e.g. centered around 0)... **simply normalize them!**

➤ Input of size (N, C, L)

N: mini-batch size, C: number of features, L: length ($=H \times W$ for images)

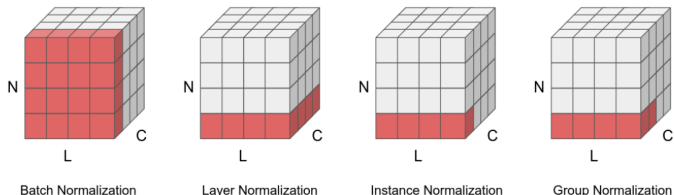


Figure: Slide from the course of Denis Coquenot

Normalization

- Compute the mean and standard deviation of the neurons activations μ and σ (for each activation a_i^l at layer l)
- Normalize the layer output $\hat{a} = (a - \mu) / \sigma$
- Can do rescaling with some learned parameters γ and β , $\gamma \hat{a} + \beta$

Normalization

- Compute the mean and standard deviation of the neurons activations μ and σ (for each activation a_i^l at layer l)
- Normalize the layer output $\hat{a} = (a - \mu) / \sigma$
- Can do rescaling with some learned parameters γ and β , $\gamma \hat{a} + \beta$

→ Simple but effective: huge help on vanishing gradients & getting smoother loss landscapes

Notes for this slide

Normalization

- Compute the mean and standard deviation of the neurons activations μ and σ (for each activation a_i^l at layer l)
- Normalize the layer output $\hat{a} := (a_i - \mu) / \sigma$
- Can do rescaling with some learned parameters γ and β , $\gamma \hat{a} + \beta$

→ Simple but effective: huge help on vanishing gradients & getting smoother loss landscapes

σ_i here refers to a standard deviation, it has nothing to do with the notation for the activation function $\sigma(x)$. Deep Learning notations can be confusing...

Similarly, the rescaling β here has nothing to do with the one from Optimizers.

The current state of the art seems to be applying BatchNorm for image tasks (convolutions) and layer norm for language tasks. Seems to come from experience rather than actual understanding.

Deep Learning Course 2: Optimization and Architectures

Still lots of works to do to make it work in practice!

1. Gradient Descent and Optimization
2. Stabilization
3. **Architectures**

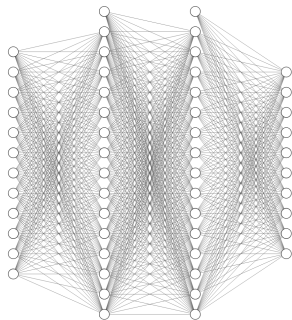
Linear or Fully Connected layer

So far we have seen the “Fully Connected” or “Feedforward” Neural Network,

$$f_{\theta}(\mathbf{x}) = f_{fc}^{(l)}(f_{fc}^{(l-1)}(\dots f_{fc}^{(2)}(f_{fc}^{(1)}(\mathbf{x}))))$$

With each layer

$$f_{fc}^{(l)}(\mathbf{x}) = \sigma(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) = \mathbf{a}^{(l)}$$



Linear or Fully Connected layer

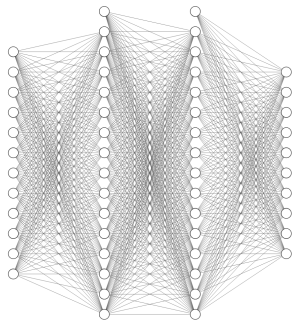
So far we have seen the “Fully Connected” or “Feedforward” Neural Network,

$$f_{\theta}(\mathbf{x}) = f_{fc}^{(L)}(f_{fc}^{(L-1)}(\dots f_{fc}^{(2)}(f_{fc}^{(1)}(\mathbf{x}))))$$

With each layer

$$f_{fc}^{(l)}(\mathbf{x}) = \sigma(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) = \mathbf{a}^{(l)}$$

$f_{fc}^{(l)}$ is called a **Linear** or **Fully Connected Layer**.



```
m = nn.Linear(20, 30)
```

Figure: Linear layer in PyTorch
(before activation)

Limitations of the Linear Layer

The Linear layer is not suited for a bunch of data types

- Images (fits into “Computer Vision”)
 - insane amount of inputs: HD = $1920 \times 1080 \times 3 > 6$ millions inputs
 - if net layer $k = 1000$, $W^{(1)}$ alone has 6 **billion** parameters

Limitations of the Linear Layer

The Linear layer is not suited for a bunch of data types

- Images (fits into “Computer Vision”)
 - insane amount of inputs: HD = $1920 \times 1080 \times 3 > 6$ millions inputs
 - if net layer $k = 1000$, $W^{(1)}$ alone has 6 **billion** parameters
 - Spatial relevance: **nearby inputs/neurons** are more likely connected

Limitations of the Linear Layer

The Linear layer is not suited for a bunch of data types

- Images (fits into “Computer Vision”)
 - insane amount of inputs: HD = $1920 \times 1080 \times 3 > 6$ millions inputs
 - if net layer $k = 1000$, $W^{(1)}$ alone has 6 **billion** parameters
 - Spacial relevance: **nearby inputs/neurons** are more likely connected
- Text (fits into “Natural Language Processing”, NLP)
 - insane amount of inputs: prompts easily have 1k words
 - for a dictionary of just 10k words, that's already 10 million inputs (one hot)

Limitations of the Linear Layer

The Linear layer is not suited for a bunch of data types

- Images (fits into “Computer Vision”)
 - insane amount of inputs: HD = $1920 \times 1080 \times 3 > 6$ millions inputs
 - if net layer $k = 1000$, $W^{(1)}$ alone has 6 **billion** parameters
 - Spacial relevance: **nearby inputs/neurons** are more likely connected
- Text (fits into “Natural Language Processing”, NLP)
 - insane amount of inputs: prompts easily have 1k words
 - for a dictionary of just 10k words, that's already 10 million inputs (one hot)
 - Contextual relevance: **words are only relate to few other words**
- Audio
 - insane amount of inputs: audio of 3 minutes ≈ 1 million inputs at 44kHz

Limitations of the Linear Layer

The Linear layer is not suited for a bunch of data types

- Images (fits into “Computer Vision”)
 - insane amount of inputs: $HD = 1920 \times 1080 \times 3 > 6$ millions inputs
 - if net layer $k = 1000$, $W^{(1)}$ alone has 6 **billion** parameters
 - Spacial relevance: **nearby inputs/neurons** are more likely connected
- Text (fits into “Natural Language Processing”, NLP)
 - insane amount of inputs: prompts easily have 1k words
 - for a dictionary of just 10k words, that's already 10 million inputs (one hot)
 - Contextual relevance: **words are only relate to few other words**
- Audio
 - insane amount of inputs: audio of 3 minutes ≈ 1 million inputs at 44kHz
 - Temporal relevance: **sounds are related to nearby sounds**
- Video (duh)

The wide family of layers

As of today there are excellent **layer choices** for most problem types

- Image: **Convolutional** Layer
- Temporal (text, audio): **Recurrent** Layer
- Temporal: **Self-attention** Layer
- Graphs: **Graph-Convolutional** Layer

The wide family of layers

As of today there are excellent **layer choices** for most problem types

- Image: **Convolutional** Layer
- Temporal (text, audio): **Recurrent** Layer
- Temporal: **Self-attention** Layer
- Graphs: **Graph-Convolutional** Layer

All layer choices f are differentiable: **backpropagation** applies

The wide family of layers

As of today there are excellent **layer choices** for most problem types

- Image: **Convolutional** Layer
- Temporal (text, audio): **Recurrent** Layer
- Temporal: **Self-attention** Layer
- Graphs: **Graph-Convolutional** Layer

All layer choices f are differentiable: **backpropagation** applies

→ everything we've seen so far (mini-batch GD, optimizers, stabilization techniques) works for all layer types.

Convolution Operation

Specialized for an image: format $H \times W \times C$, height $\times$ width $\times$ channels (e.g. RGB)

- Visualize and understand it with [this interactive website](#)
- See the [course from Alexandre Meyer](#) for many examples of filters (humanly understandable)

Next time

- Generalization: test vs train, generalization techniques
- Practical tricks: data augmentation, transfer
- Other architectures